

Clean Expression CCI Mix 开发报告

将 intra 特征从 GRN 边替换为 expression 点特征，并保留 inter CCI-only 通讯特征

2026 因果涌现项目

2026-07-02

目录

1. 开发目标	2
2. 必须确认的 lower/upper 独立性	2
3. 新方法的数学定义	4
4. 代码层面的建议实现	5
5. 接入 expression OT 和 EI 的方式	8
6. 无信息泄露验收标准	9
7. 与已有方法的实验定位	10
8. 推荐实现结论	10

本报告基于 /mnt/data/code.zip.zip 的静态代码阅读和当前讨论目标撰写。目标不是修改原有 `clean_grn_cci_expr_mix`，而是新增一个 network method: `clean_expression_cci_mix`。本报告的重点是定义新方法、确认 lower/upper 无映射与无跨层信息泄露，并说明它如何接入现有 expression OT 与 EI 计算链路。

1. 开发目标

新增 network method:

`clean_expression_cci_mix`

论文/报告中的展示名建议写成:

Clean Expression CCI Mix

这个名字的含义是:

Clean Expression CCI Mix = intra expression + inter CCI-only.

在这个公式里，Clean Expression CCI Mix 表示新的 network 特征构造方法；intra expression 表示 unit 内部不再使用 GRN 边，而是直接使用 gene expression 点特征；inter CCI-only 表示 unit 与 unit 之间仍然只使用 COMMOT 细胞通讯分数，不把 GRN 权重混入 inter 边。

当前已有方法 `clean_grn_cci_expr_mix` 的核心是:

$$\mathbf{x}_u^{(t)} = \left[\mathbf{x}_{u,\text{intra_grn_expr}}^{(t)} \mid \mathbf{x}_{u,\text{inter_cci}}^{(t)} \right].$$

这里， $\mathbf{x}_u^{(t)}$ 表示时间点 t 上 unit u 的总特征向量； u 表示一个 lower 或 upper 层级上的 unit； t 表示发育时间点； $\mathbf{x}_{u,\text{intra_grn_expr}}^{(t)}$ 表示 unit u 的表达加权 GRN 边特征； $\mathbf{x}_{u,\text{inter_cci}}^{(t)}$ 表示 unit u 的 inter CCI 通讯特征；符号 $[\cdot \mid \cdot]$ 表示按列拼接两个特征块。

新方法要改成:

$$\mathbf{x}_u^{(t)} = \left[\mathbf{x}_{u,\text{intra_expr}}^{(t)} \mid \mathbf{x}_{u,\text{inter_cci}}^{(t)} \right].$$

这里， $\mathbf{x}_{u,\text{intra_expr}}^{(t)}$ 表示 unit u 自身的 gene expression 点特征； $\mathbf{x}_{u,\text{inter_cci}}^{(t)}$ 仍然表示 unit u 的 inter CCI 通讯特征。也就是说，新方法删除 intra GRN 边建模，保留 CCI-only inter 建模。

2. 必须确认的 lower/upper 独立性

本次改动必须保持现在 `clean_grn_cci_mix` 和 `clean_grn_cci_expr_mix` 的 lower/upper 计算逻辑：lower 层级使用 lower 自己的表达和 CCI 构造特征，upper 层级使用 upper 自己的表

达和 CCI 构造特征。lower 特征不投影到 upper，upper 特征也不由 lower 聚合得到。

对任意时间点 t ，新方法应该产生两组原生 unit 特征矩阵：

$$X_{\text{lower}}^{(t)} \in \mathbb{R}^{n_{\text{lower}}^{(t)} \times D},$$

$$X_{\text{upper}}^{(t)} \in \mathbb{R}^{n_{\text{upper}}^{(t)} \times D}.$$

在第一个公式里， $X_{\text{lower}}^{(t)}$ 表示时间点 t 的 lower 层级特征矩阵； $n_{\text{lower}}^{(t)}$ 表示时间点 t 的 lower unit 数量； D 表示总特征维度。第二个公式里， $X_{\text{upper}}^{(t)}$ 表示时间点 t 的 upper 层级特征矩阵； $n_{\text{upper}}^{(t)}$ 表示时间点 t 的 upper unit 数量； D 同样表示总特征维度。

这两个矩阵的行空间是不同的。 $X_{\text{lower}}^{(t)}$ 的每一行是一个 lower unit， $X_{\text{upper}}^{(t)}$ 的每一行是一个 upper unit。它们可以共享列名，例如同样有 `intra_expr:GeneA` 和 `inter_cci:Ligand->Receptor[...]`，但它们的数值必须分别从各自层级的数据中计算。

因此，新方法的 `NetworkContext` 必须设置：

```
feature_alignment_space = "native_units"
```

这个设置非常关键。因为 `mignet_ce/representations/graph_features.py` 里有分支：当 `context.feature_alignment_space == "native_units"` 时，代码直接读取 `context.lower_mats` 和 `context.upper_mats`；只有在不是 `native_units` 时，才会调用 `aggregate_lower_features_to_upper(...)` 这种 lower-to-upper 聚合逻辑。新方法必须走 `native branch`。

对应的正确链条是：

$$X_{\text{lower}}^{(t_0)}, X_{\text{lower}}^{(t_1)} \longrightarrow P_{\text{lower}}^{(t_0 \rightarrow t_1)} \longrightarrow EI_{\text{lower}}^{(t_0 \rightarrow t_1)},$$

$$X_{\text{upper}}^{(t_0)}, X_{\text{upper}}^{(t_1)} \longrightarrow P_{\text{upper}}^{(t_0 \rightarrow t_1)} \longrightarrow EI_{\text{upper}}^{(t_0 \rightarrow t_1)}.$$

在第一条链条中， $X_{\text{lower}}^{(t_0)}$ 和 $X_{\text{lower}}^{(t_1)}$ 是两个时间点上的 lower 特征矩阵； $P_{\text{lower}}^{(t_0 \rightarrow t_1)}$ 是 lower 层级跨时间转移核； $EI_{\text{lower}}^{(t_0 \rightarrow t_1)}$ 是 lower 层级 effective information。第二条链条中， $X_{\text{upper}}^{(t_0)}$ 和 $X_{\text{upper}}^{(t_1)}$ 是两个时间点上的 upper 特征矩阵； $P_{\text{upper}}^{(t_0 \rightarrow t_1)}$ 是 upper 层级跨时间转移核； $EI_{\text{upper}}^{(t_0 \rightarrow t_1)}$ 是 upper 层级 effective information。

最终比较仍然是：

$$EI_{\text{gain}}^{(t_0 \rightarrow t_1)} = EI_{\text{upper}}^{(t_0 \rightarrow t_1)} - EI_{\text{lower}}^{(t_0 \rightarrow t_1)}.$$

这里， $EI_{\text{gain}}^{(t_0 \rightarrow t_1)}$ 表示时间对 $t_0 \rightarrow t_1$ 上 upper 相对 lower 的 EI 增益； $EI_{\text{upper}}^{(t_0 \rightarrow t_1)}$ 表示 upper 层级的 EI； $EI_{\text{lower}}^{(t_0 \rightarrow t_1)}$ 表示 lower 层级的 EI。

3. 新方法的数学定义

设 G 是 shared gene 集合。这里, G 表示所有时间点、lower 层级和 upper 层级共同可见的基因集合。设 $|G| = m$ 。这里, m 表示 shared gene 的数量。对任意基因 $g \in G$, 这里的 g 表示一个具体基因。

3.1 intra expression 特征块

对时间点 t 的 unit u , 读取 h5ad 表达矩阵中的表达值:

$$e_{u,g}^{(t)}.$$

这里, $e_{u,g}^{(t)}$ 表示时间点 t 上 unit u 中基因 g 的表达值; u 表示 lower 或 upper 层级上的一个 unit; g 表示 shared gene 集合 G 中的一个基因。

新方法的 expression 特征列建议命名为:

```
intra_expr:<gene>
```

例如:

```
intra_expr:Tbx5
intra_expr:Actc1
intra_expr:Hand2
```

对应的 expression 特征块为:

$$E^{(t)} = \left[E_{u,g}^{(t)} \right]_{u \in U^{(t)}, g \in G} \in \mathbb{R}^{n^{(t)} \times m}.$$

这里, $E^{(t)}$ 表示时间点 t 的 expression 特征矩阵; $E_{u,g}^{(t)}$ 表示 unit u 在 gene g 这个 expression 特征列上的数值; $U^{(t)}$ 表示时间点 t 的 unit 集合; $n^{(t)}$ 表示时间点 t 的 unit 数量; m 表示 shared gene 数量。

建议第一版使用现有配置 `cfg.feature_log1p` 控制 expression 块是否做 log1p。也就是:

$$E_{u,g}^{(t)} = \begin{cases} \log(1 + e_{u,g}^{(t)}), & \text{if } \text{cfg.feature_log1p} = \text{True}, \\ e_{u,g}^{(t)}, & \text{if } \text{cfg.feature_log1p} = \text{False}. \end{cases}$$

在这个公式里, $E_{u,g}^{(t)}$ 表示最终进入总特征矩阵的 expression 数值; $e_{u,g}^{(t)}$ 表示 h5ad 中读取到的原始表达值; $\log(1 + e_{u,g}^{(t)})$ 表示 log1p 变换后的表达值; `cfg.feature_log1p` 是现有配置项, 用于控制 native graph feature 是否做 log1p。这样处理的好处是 expression 块和 CCI 块使用同一个 log1p 开关, 不额外引入新的配置项。

3.2 inter CCI-only 特征块

设 ligand 为 ℓ , receptor 为 r 。这里, ℓ 表示 ligand gene 或 ligand complex; r 表示 receptor gene 或 receptor complex。对 source unit u 和 destination unit v , COMMOT 给出 CCI score:

$$s_{u,v,\ell,r}^{(t)}$$

这里, $s_{u,v,\ell,r}^{(t)}$ 表示时间点 t 上 source unit u 到 destination unit v 的 ligand-receptor 通讯强度; u 是发送信号的 unit; v 是接收信号的 unit; ℓ 是 ligand; r 是 receptor。

代码已有 `_build_commot_inter_edges(...)` 会对 COMMOT score 做 min-max normalization, 得到:

$$\tilde{s}_{u,v,\ell,r}^{(t)}$$

这里, $\tilde{s}_{u,v,\ell,r}^{(t)}$ 表示归一化后的 CCI score。新方法必须保持 `inter_influence_mode="cci_only"`, 因此 inter edge 的 `influence_score` 就是 $\tilde{s}_{u,v,\ell,r}^{(t)}$, 不再乘 GRN 权重。

对每个 source unit u , inter CCI 特征按 source unit 聚合:

$$C_{u,\ell,r}^{(t)} = \sum_{v \neq u} \tilde{s}_{u,v,\ell,r}^{(t)}$$

这里, $C_{u,\ell,r}^{(t)}$ 表示时间点 t 上 unit u 在 ligand ℓ -receptor r 这个 CCI 特征列上的数值; $v \neq u$ 表示不统计自环通讯; $\tilde{s}_{u,v,\ell,r}^{(t)}$ 表示 unit u 到 unit v 的归一化 CCI score。

对应的 CCI 特征列名沿用现有 native schema:

```
inter_cci:<ligand>-><receptor>[<lr_key>]
```

3.3 总特征矩阵

新方法最终把 expression 特征块和 CCI 特征块按列拼接:

$$X^{(t)} = [E^{(t)} \mid C^{(t)}].$$

这里, $X^{(t)}$ 表示时间点 t 的总特征矩阵; $E^{(t)}$ 表示时间点 t 的 intra expression 特征块; $C^{(t)}$ 表示时间点 t 的 inter CCI 特征块; 符号 $[E^{(t)} \mid C^{(t)}]$ 表示把 $E^{(t)}$ 和 $C^{(t)}$ 横向拼接。

如果 $E^{(t)} \in \mathbb{R}^{n^{(t)} \times m}$, $C^{(t)} \in \mathbb{R}^{n^{(t)} \times q}$, 那么:

$$X^{(t)} \in \mathbb{R}^{n^{(t)} \times (m+q)}.$$

这里, q 表示 CCI feature 的数量; $m + q$ 表示 expression 特征和 CCI 特征拼接后的总维度。

4. 代码层面的建议实现

4.1 新增 network 文件

新增文件:

```
mignet_ce/networks/clean_expression_cci_mix.py
```

新增类:

```
class CleanExpressionCCIMixBuilder(CleanGRNCCIMixBuilder):
    network_method = "clean_expression_cci_mix"
```

可以继承 CleanGRNCCIMixBuilder 以复用以下通用逻辑:

- `_compute_shared_expression_genes(...)`
- `_stable_upper_units(...)`
- overlap / correspondence / coverage 相关构造逻辑
- NetworkContext 返回结构

但是不要复用 CleanGRNCCIMixBuilder.build_pair_context(...) 的 intra GRN matrix 构造部分。新类应写自己的 build_pair_context(...), 因为它需要把 expression matrix 和 CCI matrix 横向拼接。

4.2 建议新增 CCI-only graph helper

当前 build_layer_graph(...) 即使设置 include_intra_grn=False, 也会读取 paths.grn_edges。这对 clean_expression_cci_mix 语义上不理想, 因为新方法不应该依赖 GRN 文件。建议在 mignet_ce/graph/builder.py 中新增 helper:

```
def build_layer_cci_graph(...):
    ...
```

这个 helper 只读取 expression h5ad、COMMOT manifest、COMMOT index 和 COMMOT by-LR sparse matrix, 不读取 GRN。它内部仍然可以复用 _build_commot_inter_edges(...), 但传入:

```
pair_lookup={}
inter_influence_mode="cci_only"
inter_grn_pair_policy="zero_if_missing"
```

因为在 cci_only 模式下, pair_lookup 不参与 influence_score 的计算。这样可以彻底避免新方法对 GRN 文件的隐性依赖。

4.3 新 builder 的核心矩阵拼接逻辑

对每个时间点 stage:

```
lower_expr = read_expression_h5ad(lower_paths.h5ad)
upper_expr = read_expression_h5ad(upper_paths.h5ad)

lower_graph = build_layer_cci_graph(... lower_expr ...)
upper_graph = build_layer_cci_graph(... upper_expr ...)
```

```

lower_expr_mat = build_expression_block(lower_expr, lower_graph.units,
    ↪ shared_genes, cfg.feature_loglp)
upper_expr_mat = build_expression_block(upper_expr, upper_graph.units,
    ↪ shared_genes, cfg.feature_loglp)

lower_cci_mat = build_native_graph_matrix(lower_graph, cci_schema,
    ↪ feature_loglp=cfg.feature_loglp)
upper_cci_mat = build_native_graph_matrix(upper_graph, cci_schema,
    ↪ feature_loglp=cfg.feature_loglp)

lower_mat = np.hstack([lower_expr_mat, lower_cci_mat])
upper_mat = np.hstack([upper_expr_mat, upper_cci_mat])

```

这个伪代码里的 `lower_expr_mat` 是 lower 层级自己的 expression 特征矩阵；`upper_expr_mat` 是 upper 层级自己的 expression 特征矩阵；`lower_cci_mat` 是 lower 层级自己的 CCI 特征矩阵；`upper_cci_mat` 是 upper 层级自己的 CCI 特征矩阵。`np.hstack(...)` 只在同一个层级、同一个时间点内部拼接 expression 和 CCI，不跨层拼接，不跨时间拼接。

4.4 feature schema

最终 feature names 应该是：

```

expr_feature_names = [f"intra_expr:{gene}" for gene in shared_genes]
feature_names = expr_feature_names + cci_schema.feature_names

```

最终 feature blocks 应该是：

```

feature_blocks = {
    "intra_expr": expr_feature_names,
    "inter_cci": cci_schema.feature_blocks.get("inter_cci", []),
}

```

这里的 `intra_expr` 是新 block，表示 unit 内部表达状态；`inter_cci` 是现有 block，表示 unit 对其他 unit 的 CCI 通讯输出。

4.5 registry 注册

修改：

```
mignet_ce/networks/registry.py
```

加入：

```

from mignet_ce.networks.clean_expression_cci_mix import
    ↪ CleanExpressionCCIMixBuilder

```

并在 `NETWORK_BUILDERS` 中加入：

```
"clean_expression_cci_mix": CleanExpressionCCIMixBuilder,
```

这样命令行就可以通过:

```
--network-method clean_expression_cci_mix
```

调用新方法。

5. 接入 expression OT 和 EI 的方式

新方法不需要改 `expr_ot.py`、`_ot_common.py` 和 `metrics.py`。原因是新方法最终仍然返回 `NetworkContext.lower_mats` 和 `NetworkContext.upper_mats`。

对时间对 $t_0 \rightarrow t_1$, expression OT 会分别计算 lower 和 upper 的 cost matrix:

$$D_{ij,lower} = 1 - \frac{\mathbf{x}_{i,lower}^{(t_0)} \cdot \mathbf{x}_{j,lower}^{(t_1)}}{\|\mathbf{x}_{i,lower}^{(t_0)}\|_2 \|\mathbf{x}_{j,lower}^{(t_1)}\|_2},$$

$$D_{ij,upper} = 1 - \frac{\mathbf{x}_{i,upper}^{(t_0)} \cdot \mathbf{x}_{j,upper}^{(t_1)}}{\|\mathbf{x}_{i,upper}^{(t_0)}\|_2 \|\mathbf{x}_{j,upper}^{(t_1)}\|_2}.$$

在第一个公式里, $D_{ij,lower}$ 表示 lower 层级 source unit i 到 lower 层级 target unit j 的 feature cost; $\mathbf{x}_{i,lower}^{(t_0)}$ 表示时间点 t_0 的第 i 个 lower unit 的特征向量; $\mathbf{x}_{j,lower}^{(t_1)}$ 表示时间点 t_1 的第 j 个 lower unit 的特征向量。第二个公式里, $D_{ij,upper}$ 表示 upper 层级 source unit i 到 upper 层级 target unit j 的 feature cost; $\mathbf{x}_{i,upper}^{(t_0)}$ 和 $\mathbf{x}_{j,upper}^{(t_1)}$ 分别表示两个时间点上的 upper unit 特征向量。

然后 OT 分别得到:

$$P_{lower}^{(t_0 \rightarrow t_1)} \quad \text{and} \quad P_{upper}^{(t_0 \rightarrow t_1)}.$$

这里, $P_{lower}^{(t_0 \rightarrow t_1)}$ 是 lower 层级的 row-stochastic transition kernel; $P_{upper}^{(t_0 \rightarrow t_1)}$ 是 upper 层级的 row-stochastic transition kernel。

EI 公式保持不变:

$$EI(P) = H(J) - H(J | I).$$

这里, $EI(P)$ 表示转移矩阵 P 的 effective information; I 表示源时间点 unit 随机变量; J 表示目标时间点 unit 随机变量; $H(J)$ 表示目标 unit 的边缘熵; $H(J | I)$ 表示给定源 unit 后目标 unit 的条件熵。

6. 无信息泄露验收标准

本方法实现完成后，应检查以下条件。

第一，`NetworkContext.feature_alignment_space` 必须等于：

```
native_units
```

第二，`mignet_ce/representations/graph_features.py` 必须走如下分支：

```
if context.feature_alignment_space == "native_units":
    lower_raw = [np.asarray(matrix, dtype=float) for matrix in context.lower_mats]
    upper_raw = [np.asarray(matrix, dtype=float) for matrix in context.upper_mats]
```

而不能走：

```
aggregate_lower_features_to_upper(...)
```

第三，new builder 中不能出现如下行为：

```
lower_mat = aggregate_lower_to_upper(...)
upper_mat = ...
```

第四，lower 的 expression matrix 必须来自 lower h5ad：

$$E_{\text{lower}}^{(t)} = F\left(\text{h5ad}_{\text{lower}}^{(t)}\right).$$

这里， $E_{\text{lower}}^{(t)}$ 表示 lower 的 expression 特征矩阵； $F(\cdot)$ 表示 expression 特征构造函数； $\text{h5ad}_{\text{lower}}^{(t)}$ 表示时间点 t 的 lower h5ad 文件。

第五，upper 的 expression matrix 必须来自 upper h5ad：

$$E_{\text{upper}}^{(t)} = F\left(\text{h5ad}_{\text{upper}}^{(t)}\right).$$

这里， $E_{\text{upper}}^{(t)}$ 表示 upper 的 expression 特征矩阵； $\text{h5ad}_{\text{upper}}^{(t)}$ 表示时间点 t 的 upper h5ad 文件。

第六，lower 的 CCI matrix 必须来自 lower 的 COMMOT 文件；upper 的 CCI matrix 必须来自 upper 的 COMMOT 文件。也就是说：

$$C_{\text{lower}}^{(t)} = G\left(\text{COMMOT}_{\text{lower}}^{(t)}\right),$$

$$C_{\text{upper}}^{(t)} = G\left(\text{COMMOT}_{\text{upper}}^{(t)}\right).$$

这里， $C_{\text{lower}}^{(t)}$ 是 lower 的 CCI 特征矩阵； $C_{\text{upper}}^{(t)}$ 是 upper 的 CCI 特征矩阵； $G(\cdot)$ 表示 CCI-only 特征构造函数。

第七，表达特征和 CCI 特征可以共享列名 schema，但不能共享行空间。共享列名只是为了保证 lower 和 upper 都有同样的 feature dimension；共享行空间才会造成 lower/upper 映射嫌疑。

新方法只共享列名，不共享行空间。

7. 与已有方法的实验定位

建议把方法关系写成下表。

method	intra block	inter block	是否用 GRN	是否用 CCI	科学问题
expression_only	gene expression	无	否	否	纯表达能否产生 EI gain
clean_grn_cci_expr_mix	GRN edge weight	CCI-only	是	是	GRN 拓扑加 CCI 是否有效
clean_grn_cci_expr_mix	GRN edge weight × expression activity	CCI-only	是	是	活跃 GRN 加 CCI 是否更有效
clean_expression_cci_mix	gene expression	CCI-only	否	是	只用表达状态加 CCI 是否足够

这个新方法的关键科学解释是：如果 clean_expression_cci_mix 明显优于 expression_only，说明 inter CCI 信息对 EI gain 有贡献。如果 clean_grn_cci_expr_mix 又明显优于 clean_expression_cci_mix，说明 GRN 边结构提供了超越表达本身的信息。如果二者接近，则需要谨慎解释当前 GRN 部分的贡献，可能 EI gain 主要来自 expression + CCI。

8. 推荐实现结论

最终建议如下：

1. 新方法命名为 clean_expression_cci_mix，论文展示名为 Clean Expression CCI Mix。
2. 新增 mignet_ce/networks/clean_expression_cci_mix.py，不要改原来的 clean_grn_cci_expr_mix.py。
3. 新方法采用路线 B：直接拼接 expression matrix 和 CCI matrix，不把 expression 伪装成 GRN self-loop。
4. intra feature column 使用 intra_expr:<gene>。
5. inter feature column 沿用 inter_cci:<ligand>-><receptor>[<lr_key>]。
6. 新方法设置 feature_alignment_space="native_units",保持 lower 和 upper 原生 unit 行空间。
7. lower 特征只由 lower h5ad 和 lower COMMOT 计算；upper 特征只由 upper h5ad 和 upper COMMOT 计算。
8. 后续 expression OT、PIJ 和 EI 计算不用改，直接复用现有 pipeline。

9. 为避免隐性 GRN 依赖，建议新增 CCI-only graph helper，不要让新方法调用会读取 GRN 文件的 `build_layer_graph(...)`。
10. 验收时检查导出的 raw native feature CSV: lower 行数必须等于 lower units 数，upper 行数必须等于 upper units 数，且 `metadata.feature_alignment_space` 必须为 `native_units`。